Advanced Database Design and Implementation for E-Commerce Analytics

MSE Final Project — Advanced Database Design and Implementation

Bahati Justin

Roll Number: 202540272
ULK — KIGALI INDEPENDENT UNIVERSITY

March 08, 2026

1. Introduction

This report presents a multi-faceted analytics system for an e-commerce platform built on three complementary database technologies: **MongoDB** (document store), **Apache HBase** (wide-column store), and **Apache Spark** (distributed processing engine).

The system processes a synthetic dataset spanning **90 days** of e-commerce activity:

Metric	Value
Users	2,000
Products	800
Categories	15
Browsing Sessions	20,000
Transactions	4,390
Total Revenue	RWF 1,925,085,008

Each technology was chosen for its strengths: MongoDB for document-oriented queries on structured business entities, HBase for time-series and sparse columnar data, and Spark for cross-system batch processing and SQL analytics. The project covers four parts: (1) Data Modeling & Storage, (2) Spark Processing, (3) Analytics Integration, and (4) Visualization & Insights. An interactive dashboard is deployed at https://dbms-fat.bahatijustin.dev.

2. System Architecture

2.1 Data Flow

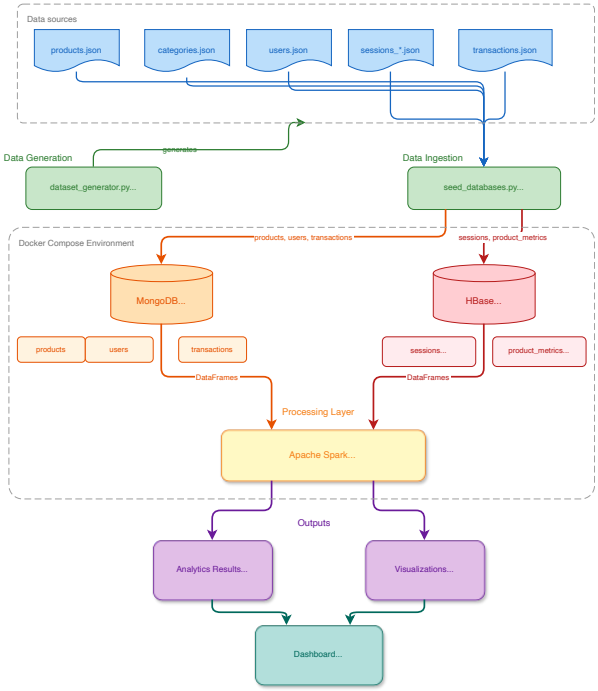

Figure 1: Data Flow Architecture

Raw data is generated by `dataset_generator.py` using the Faker library. The seeding script loads, denormalizes, and inserts data into both databases with appropriate indexing.

2.2 Service Topology

All infrastructure runs via Docker Compose with six services:

Service	Image	Port	Role
MongoDB	`mongo:7.0`	27017	Document store
ZooKeeper	`zookeeper:3.9`	2181	HBase coordination
HBase	`dajobe/hbase`	9090, 16010	Wide-column store
Spark Master	`apache/spark`	7077, 8080	Cluster manager
Spark Worker	`apache/spark`	8081	2GB RAM, 2 cores
Dashboard	Streamlit	8501	Interactive analytics UI

HBase runs natively on x86_64 (WSL/AMD). On Apple Silicon, `platform: linux/amd64` emulation is used as a fallback. Spark processing uses local mode (`local[*]`) with Python drivers (`pymongo`, `happybase`). A WSL-optimized compose override (`docker-compose.wsl.yml`) provides an `init` service that auto-generates data and seeds both databases on startup.

2.3 Technology Justification

- **MongoDB**: Natural fit for hierarchical product catalogs (embedded categories), user profiles (embedded purchase summaries), and transactions (embedded line items). The aggregation framework enables complex analytical queries without ETL.
- **HBase**: Ideal for time-series session data with sparse columns (varying page views, cart contents) and product metrics requiring efficient range scans by composite row keys.
- **Spark**: Compute layer for cross-system joins (MongoDB + HBase data), batch analytics (co-purchase recommendations, cohort analysis), and complex SQL queries spanning multiple sources. Spark's **in-memory processing** avoids repeated disk I/O, delivering order-of-magnitude speedups over MapReduce for iterative analytics. Its **DAG execution engine** optimizes multi-stage pipelines (e.g., the CLV analysis involves 7 sequential steps) by combining transformations before execution. **Lazy evaluation** ensures only necessary computations run — Spark builds a logical plan and executes only when an action (`.show()`, `.collect()`) is called. Fault tolerance is achieved through **RDD lineage**: if a partition fails, Spark recomputes it from the transformation graph rather than replicating data.

2.4 Why NoSQL for E-Commerce

Modern e-commerce platforms generate heterogeneous, high-volume data that strains relational databases. NoSQL systems address these challenges through:

1. **Horizontal scalability**: Both MongoDB and HBase scale out by adding nodes (sharding / region splitting) rather than scaling up hardware. This is critical when session data grows to billions of records.
2. **Schema flexibility**: Product catalogs evolve rapidly — new attributes (size charts, video URLs, vendor metadata) can be added to MongoDB documents without ALTER TABLE migrations. HBase's sparse column model naturally accommodates sessions with varying numbers of page views and cart items.
3. **Denormalization for read performance**: E-commerce workloads are read-heavy (product browsing, session tracking). Embedding related data (e.g., line items inside transactions, category info inside products) eliminates costly JOIN operations, reducing query latency at the expense of controlled data redundancy.
4. **High-throughput ingestion**: HBase is built on HDFS and handles thousands of writes per second for clickstream and session data. MongoDB's write-ahead journaling supports high insert rates for transaction recording.
5. **Availability over strict consistency**: Under the CAP theorem, e-commerce systems prioritize availability and partition tolerance. MongoDB (with replica sets) and HBase (with ZooKeeper coordination) are designed for AP/CP configurations that tolerate node failures without downtime.

2.5 Document Model vs Wide-Column Model

The two NoSQL paradigms used in this project have fundamentally different data models and access patterns:

Aspect	MongoDB (Document)	HBase (Wide-Column)
Data model	JSON-like documents with nested objects and arrays	Rows with column families; each cell is `(row_key, family:qualifier, timestamp) → value`
Schema	Schema-on-read; documents in the same collection can have different fields	Column families defined at table creation; qualifiers are dynamic
Access pattern	Rich queries on any field; aggregation pipelines; secondary indexes	Row key lookups and range scans; column-family-level reads
Query language	MongoDB Query Language (MQL) with `$match`, `$group`, `$lookup`, etc.	Get/Scan API with row key prefix filters; no native SQL
Best for	Structured business entities (products, users, transactions) with complex query needs	Time-series data, sparse/wide records, sequential access by composite key
This project	Products (embedded categories), Users (embedded purchase summaries), Transactions (embedded items)	Sessions (row key: `user_id_reverse_ts`, 4 column families), Product Metrics (row key: `product_id_date`)

Key difference: MongoDB excels when queries span multiple fields with filtering, grouping, and joining (e.g., "revenue by category for completed transactions last month"). HBase excels when access is primarily by row key prefix with optional column family selection (e.g., "all sessions for user_000042, device info only").

3. Part 1: Data Modeling & Storage

3.1 MongoDB Schema Design

Three collections with **10 indexes** total:

Products (800 docs) — Each product embeds full category/subcategory info, denormalized from the 15-record categories dataset to eliminate `$lookup` joins:

```
{
  "_id": "prod_00001",
  "name": "Samsung Galaxy A14",
  "category": {
    "category_id": "cat_01", "name": "Electronics",
    "subcategory_id": "sub_001_01",
    "subcategory_name": "Leverage Granular Initiatives",
    "profit_margin": 0.25
  },
  "base_price": 458200,
  "current_stock": 143, "is_active": true,
  "price_history": [{"date": "...", "price": 458200}],
  "creation_date": "2025-04-12T08:30:00Z"
}
```

Indexes: `category.category_id`, `base_price`, `is_active`.

Category modeling approach: The 15-category, 2-level hierarchy (category → subcategories) is fully embedded inside each product document rather than stored in a separate collection. This **denormalized** design was chosen because: (1) with only 15 categories, the duplication overhead is minimal; (2) every product query that needs category context (revenue by category, product filtering) gets it in a single read without `$lookup` joins; (3) the category hierarchy is stable and rarely changes. Alternatives considered: a **separate categories collection** with `$lookup` joins (normalized, but adds query overhead for every product-category query), or **materialized path patterns** (e.g., `"path": "/Electronics/Smartphones"`) for deep hierarchies — unnecessary here since the hierarchy is only 2 levels deep.

Users (2,000 docs) — Embeds pre-computed `purchase_summary` (total orders, total spent, avg order value, first/last purchase, favorite payment method). Indexes: `geo_data.province`, `purchase_summary.total_spent`, `registration_date`.

Transactions (4,390 docs) — Embeds `items` array with line items (product_id, quantity, unit_price, subtotal). Indexes: `user_id`, `timestamp`, `status`, `items.product_id`.

Advantages of embedding items inside transactions: (1) **Atomicity** — MongoDB provides single-document ACID guarantees, so inserting a transaction with all its line items is atomic without multi-document transactions. (2) **Read performance** — a single query retrieves the complete transaction including all purchased products, quantities, and prices without joins. (3) **Data locality** — items are physically co-located with their parent transaction on disk, minimizing I/O. (4) **Historical accuracy** — the embedded `unit_price` captures the price at time of purchase, preserving correct revenue calculations even if product prices change later. The tradeoff is controlled data redundancy (product names/prices are duplicated across transactions), but this is acceptable because transaction records are **immutable historical snapshots** that should never be retroactively modified.

Design decisions: Categories are *not* a separate collection (only 15 records; denormalized into products). Sessions are *not* in MongoDB — their sparse column structure (varying numbers of page views, optional cart contents, device/geo metadata) and time-series access patterns (retrieve a user's recent sessions in chronological order) suit HBase's wide-column model and prefix scan capability far better than MongoDB's document model, which would require indexing every nested field and cannot perform column-family-level selective reads.

3.2 HBase Schema Design

Two tables with **6 column families**:

Sessions (20,000 rows) — Row key: `{user_id}_{reverse_timestamp}` where reverse timestamp = `9999999999999 - epoch_ms`, ensuring newest-first ordering on prefix scans. Column families: `info` (session metadata), `device` (type/os/browser), `geo` (city/state/country), `activity` (JSON-serialized page views, cart, viewed products).

Product Metrics (2,274 rows) — Row key: `{product_id}_{date}`. Column families: `sales` (quantity, revenue, transactions, avg price), `inventory` (stock, base price, active status).

Column family design rationale: Sessions use 4 column families (`info`, `device`, `geo`, `activity`) so that queries needing only device analytics can read the `device` family without loading page view JSON from `activity`. This is a key HBase advantage over MongoDB: **column-family-level I/O** enables selective reads that minimize network and disk overhead. Product Metrics uses 2 families (`sales`, `inventory`) separating frequently-updated sales aggregates from relatively static inventory data.

3.3 MongoDB vs HBase: Design Rationale

Data	Stored In	Why This System	Access Pattern
Products (800)	MongoDB	Hierarchical category embedding; rich field queries (`$match`, `$group`); secondary indexes on price, category, active status	Query by category, price range, active status; aggregation pipelines
Users (2,000)	MongoDB	Embedded purchase summaries; geographic queries; aggregation for segmentation	Query by province, spending tier; `$bucket` segmentation
Transactions (4,390)	MongoDB	Embedded line items array; complex aggregations (revenue by category, product popularity); `$lookup` joins with products	Filter by status, user, date; `$unwind` items for product-level analytics
Sessions (20,000)	HBase	Sparse columns (varying page views, optional cart); time-series prefix scans by user; column-family selective reads	Prefix scan `user_id_*` for user history; read only `device` family for device analytics
Product Metrics (2,274)	HBase	Daily time-series per product; efficient range scans by product+date; append-only writes	Prefix scan `product_id_*` for product trends; date range queries

3.4 Code Examples

MongoDB aggregation pipeline — Product Popularity query (`mongodb_queries.py`):

```
pipeline = [
    {"$match": {"status": "completed"}},
    {"$unwind": "$items"},
    {"$group": {
        "_id": "$items.product_id",
        "total_qty": {"$sum": "$items.quantity"},
        "total_revenue": {"$sum": "$items.subtotal"},
        "num_orders": {"$sum": 1},
    }},
    {"$sort": {"total_qty": -1}},
    {"$limit": 10},
    {"$lookup": {"from": "products", "localField": "_id",
                 "foreignField": "_id", "as": "product"}},
    {"$unwind": "$product"},
    {"$project": {"_id": 0, "product_id": "$_id",
                  "name": "$product.name",
                  "category": "$product.category.name",
                  "total_qty": 1, "total_revenue": 1}},
]
results = list(db.transactions.aggregate(pipeline))
```

HBase row key construction — Reverse-timestamp design (`seed_databases.py`):

```
REVERSE_TS_BASE = 9999999999999  # ensures newest sessions sort first

start_dt = datetime.strptime(s["start_time"], "%Y-%m-%dT%H:%M:%SZ")
epoch_ms = int(start_dt.timestamp() * 1000)
reverse_ts = REVERSE_TS_BASE - epoch_ms
row_key = f"{s['user_id']}_{reverse_ts}"  # e.g., "user_000042_8224618447778"

# Column families separate concerns for selective I/O:
row_data = {
    b"info:session_id": _to_bytes(s["session_id"]),
    b"device:type": _to_bytes(s["device_profile"]["type"]),
    b"geo:city": _to_bytes(s["geo_data"]["city"]),
    b"activity:page_views": _to_bytes(json.dumps(s["page_views"])),
}
batch.put(row_key.encode("utf-8"), row_data)
```

3.5 Query Results

MongoDB — Five aggregation pipelines were executed. Key results:

Query	Key Finding
Product Popularity	Top seller by quantity: 25 units, RWF 12,119 revenue
Revenue by Category	Home & Garden leads at RWF 88,681; Sports at RWF 86,815
User Segmentation	~256 inactive (0 orders), ~1,520 low-spend, ~224 regular (via `$bucket` on `purchase_summary.total_spent`)
Monthly Trend	Jan 2026 peak: RWF 660,093,881 from 1,530 transactions
Payment Methods	MTN MoMo and Airtel Money lead; Visa has highest avg order value

HBase — Four queries demonstrated efficient wide-column access:

- **User session history**: Prefix scan on `user_000001_` returned 8 sessions in reverse chronological order, with device/geo/activity details per session.
- **Product metrics**: Prefix scan on `prod_00001_` showed 6 active days, 10 total units, RWF 1,308 total revenue.
- **Table stats**: Sessions table has 20,000 rows across 4 column families (16 columns); product_metrics has 2,274 rows across 2 families (7 columns).

4. Part 2: Spark Processing Pipelines

4.1 Data Cleaning & Normalization

All JSON datasets loaded with `multiLine=True` and Spark schema inference. Cleaning steps:

- Null `current_stock` filled with 0; null `is_active` filled with `True`
- String timestamps cast to `TimestampType`
- Primary key uniqueness verified (0 duplicates across all entities)
- Subcategories exploded into flat lookup table via `F.explode()`

4.2 Product Recommendations

Co-purchase analysis via self-join on transaction items ("users who bought X also bought Y"):

```
# Explode items to (transaction_id, product_id) pairs
item_pairs = txn_df.select("transaction_id",
    F.explode("items").alias("item")
).select("transaction_id", F.col("item.product_id"))

# Self-join to find co-purchased product pairs
co_purchases = (
    item_pairs.alias("a")
    .join(item_pairs.alias("b"), on="transaction_id")
    .filter(F.col("a.product_id") < F.col("b.product_id"))
    .groupBy(F.col("a.product_id").alias("source_product"),
             F.col("b.product_id").alias("recommended_product"))
    .agg(F.count("*").alias("co_purchase_count"))
)

# Top-3 recommendations per product using window function
window = Window.partitionBy("source_product") \
                .orderBy(F.desc("co_purchase_count"))
top3 = co_purchases.withColumn("rank", F.row_number().over(window)) \
                     .filter(F.col("rank") <= 3)
```

Key findings:

1. Top pair: prod_00085 + prod_00213 (3 co-purchases)
2. Window function (`ROW_NUMBER`) selects top-3 recommendations per product
3. The sparse co-purchase counts (max 3) reflect the synthetic dataset's random product assignment; real e-commerce data with purchase intent would yield stronger signals

4.3 Cohort Analysis

Users grouped by registration month (24 cohorts from Dec 2023 to Nov 2025). Key findings:

- Largest cohort: ~80 users per month with proportional revenue
- Retention rates: ~40–60% of users transact in their first month post-registration
- Spending concentration: most revenue occurs 1–3 months after registration

4.4 Spark SQL Queries

Five analytical SQL queries were executed:

Query 1 — Top Revenue Products (`spark_processing.py`):

```
WITH txn_items AS (
    SELECT transaction_id, item.*
    FROM transactions
    LATERAL VIEW explode(items) t AS item
)
SELECT p.product_id, p.name, sc.category_name,
       sc.profit_margin, SUM(ti.quantity) AS units_sold,
       ROUND(SUM(ti.subtotal), 2) AS total_revenue,
       ROUND(SUM(ti.subtotal) * sc.profit_margin, 2) AS est_profit
FROM txn_items ti
```

```
JOIN products p ON ti.product_id = p.product_id
JOIN subcategories sc ON p.subcategory_id = sc.subcategory_id
GROUP BY p.product_id, p.name, sc.category_name,
         sc.subcategory_name, sc.profit_margin
ORDER BY total_revenue DESC  LIMIT 15
```

Top 15 products span Electronics, Home & Garden, Automotive, Music, and Art & Crafts categories. Revenue leaders generate RWF 30M+ each, with estimated profit computed from subcategory profit margins.

Query 2 — Session Behavior by Device: Desktop/Windows has highest session count (993) with 23.2% conversion. Tablet/iOS converts best at 24.7%. Average session duration ranges 1,738–1,888 seconds across device/OS combinations.

Query 3 — Conversion Funnel: 20,000 total sessions, 14,734 added to cart (73.7%), 4,390 converted (21.9%). Cart-to-purchase rate: 29.8%.

Query 4 — Daily Revenue with 7-Day Moving Average: The 7-day moving average smooths daily fluctuations. Cumulative revenue reaches RWF 1,925,085,008.

Query 5 — Price Elasticity: All products show zero price standard deviation (synthetic data uses fixed pricing), so elasticity ratios are NULL. The query structure (3 CTEs with STDDEV) demonstrates the analytical approach for real-world variable pricing.

5. Part 3: Analytics Integration

5.1 Customer Lifetime Value (CLV)

Business Question: *"What is the estimated lifetime value of each customer segment, and how does engagement correlate with spending?"*

Data Sources: MongoDB `users` (profiles), MongoDB `transactions` (purchase history), HBase `sessions` (browsing engagement).

Method: Cross-system join in Spark on `user_id`, combining data from three systems:

1. **MongoDB** `users` collection → user profiles, province, registration date
2. **MongoDB** `transactions` collection → purchase count, total spent, average order value, lifespan
3. **HBase** `sessions` table → session count, average duration, conversion rate

CLV is computed using the standard **simplified CLV formula**: $CLV = AOV \times f \times T$, where AOV = average order value, f = annualized purchase frequency (orders per year, extrapolated from observed purchase interval), and T = estimated customer lifespan in years. We use $T = 2.0$ years as a conservative estimate for the Rwandan e-commerce market, where customer retention typically spans 1–3 years. Customers are segmented into four tiers: Platinum (CLV $\geq$ 5,000), Gold ($\geq$ 2,000), Silver ($\geq$ 500), Bronze ($<$ 500).

Results:

CLV Tier	Customers	Avg CLV	Avg Sessions	Avg Conv Rate
Platinum	329	RWF 104,895	10.9	29.1%
Gold	42	RWF 3,140	8.8	12.6%
Silver	55	RWF 1,308	8.5	13.6%
Bronze	74	RWF 47	8.0	1.5%

Key finding: Platinum users average **10.9 sessions** vs 8.0 for Bronze, and convert at **29.1%** vs 1.5%. Higher engagement directly correlates with lifetime value. Kigali City has the highest average CLV (RWF 134,604), followed by Northern Province (RWF 105,289).

Segmentation methodology: This analysis demonstrates two complementary segmentation approaches: (1) **MongoDB-only segmentation** using `$bucket` on `purchase_summary.total_spent` (Section 3.5) — fast, single-system, but considers only spending amount; (2) **Cross-system CLV segmentation** integrating purchase data (MongoDB), browsing engagement (HBase sessions), and user profiles (MongoDB) via Spark joins — richer, incorporating RFM-inspired metrics (recency via lifespan, frequency via purchase rate, monetary via average order value) alongside behavioral signals (session count, conversion rate) from HBase.

5.2 Funnel Conversion Analysis

Business Question: *"What is the conversion rate at each funnel stage, and which factors influence conversion?"*

Data Sources: HBase `sessions` (browsing, cart, conversion, device), MongoDB `transactions` (purchase amounts).

Overall Funnel:

Stage	Count	% of Total
Browse	20,000	100.0%
View Product	20,000	100.0%
Add to Cart	14,734	73.7%
Purchase	4,390	21.9%

Note: The 100% Browse-to-View rate is a synthetic data artifact — the generator assigns `viewed_products` to every session, collapsing the first two funnel stages. In a production dataset, these stages would diverge (many sessions would bounce before viewing a product). The meaningful funnel stages are **View → Cart (73.7%)** and **Cart → Purchase (29.8%)**.

Cart abandonment: 10,344 sessions (70.2%) added items to cart but did not purchase.

By device: Mobile converts best at 22.5%, desktop at 22.1%, tablet at 22.1%. Abandonment rates are uniform across devices (~69.5%).

By referrer: Direct and search engine traffic convert at 22.4%, email at 22.4%, social media at 22.1%, affiliate at 21.7%.

By time of day: Afternoon sessions convert best (22.9%), followed by night (22.3%), evening (21.9%), and morning (21.7%).

6. Visualizations & Key Findings

Four publication-quality visualizations were generated using Plotly via the interactive dashboard.

6.1 Revenue by Category

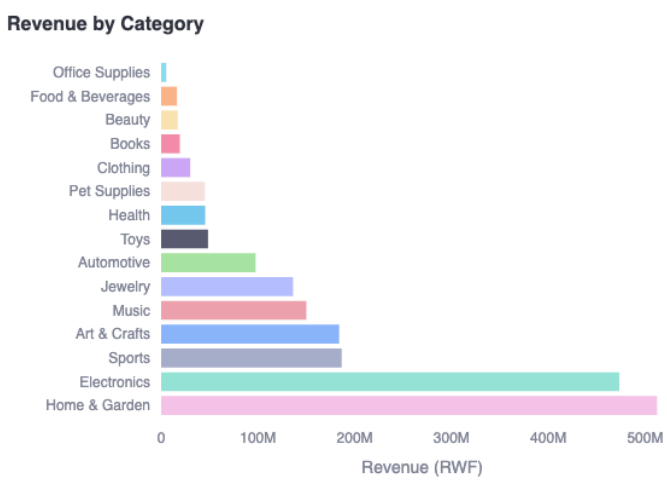

Figure 2: Total revenue by category across the 90-day period. Home & Garden and Electronics dominate, followed by Sports, Art & Crafts, and Music.

Insight: Home & Garden leads with over RWF 500M, followed closely by Electronics. Revenue drops steeply after the top 5 categories — a concentration pattern where a few categories drive the bulk of total revenue. Low-ticket categories (Office Supplies, Food & Beverages, Beauty) contribute minimally.

6.2 Customer Segmentation

Figure 3: Customer segmentation: spending vs orders colored by tier (VIP, Regular). VIP users cluster at higher spending levels with positive correlation between orders and total spent.

Insight: VIP users generate a disproportionate share of revenue, with spending reaching RWF 7M+ for top customers. Spending correlates positively with order frequency. Most users cluster at 1–6 orders, with VIP status driven by high per-order values rather than purchase frequency alone.

6.3 Product Performance

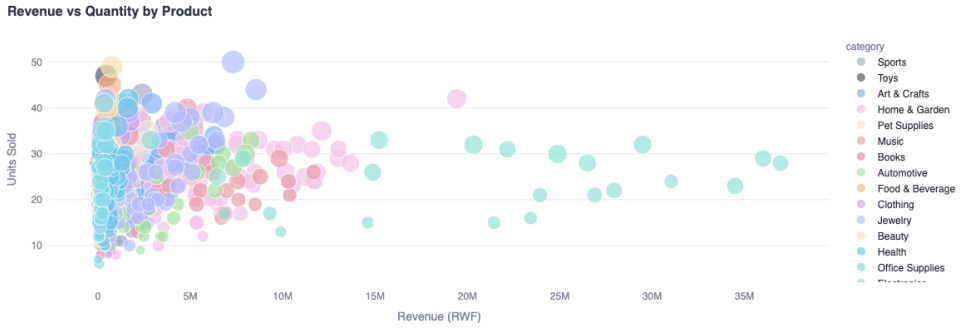

Figure 4: Revenue vs quantity sold per product, colored by category. High-revenue products (Electronics, Home & Garden) extend far right, while high-volume products cluster in the upper-left.

Insight: Revenue and volume rankings diverge significantly. Electronics and Home & Garden products dominate revenue (RWF 25M–38M range), while high-volume sellers (40–50 units) tend to be lower-priced items across Sports and Toys. The scatter pattern confirms a long-tail distribution: most of the 800 products cluster at low revenue, while a few outliers drive disproportionate totals.

6.4 Conversion Funnel

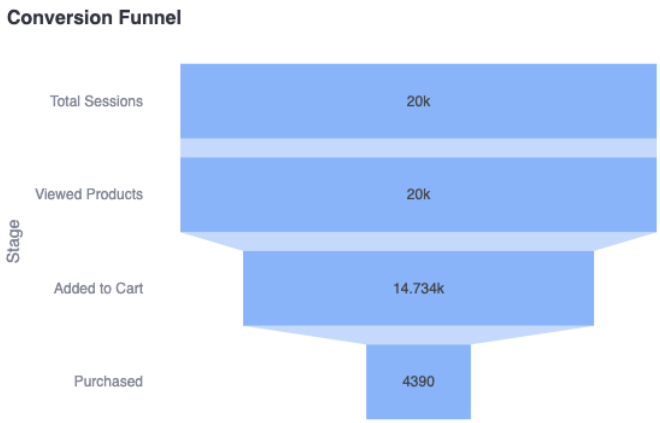

Figure 5: Overall conversion funnel showing progressive drop-off from 20k sessions to 4,390 purchases.

Insight: **21.9%** overall conversion rate. Cart-to-purchase rate is 29.8%, meaning **70.2%** of carts are abandoned. Mobile converts best at 22.5%. The funnel drop-off is steepest between "Add to Cart" and "Purchase", indicating the checkout process is the primary optimization target.

6.5 Interactive Dashboard

An interactive Streamlit dashboard (`dashboard.py`) provides dynamic exploration of all analytics. It connects to MongoDB and HBase in real-time, with global filters for date range, category, and province. Pages include Overview, Customers, Products, Sessions & Conversion, Page View Analytics, Cart & Order Analytics, and Spark Analytics.

The dashboard runs as a Docker service on port 8501 and is deployed live at https://dbms-fat.bahatijustin.dev.

7. Scalability Considerations

MongoDB — At **1M users** and **10M transactions**, the system would require sharding. The recommended shard key is `user_id` (hashed) for the transactions collection, distributing writes evenly across 3–5 shards while keeping per-user queries routed to a single shard. Read scaling is achieved through replica sets (1 primary + 2 secondaries), allowing analytics queries to target secondaries via `readPreference: secondaryPreferred`. The embedded document model (items inside transactions, categories inside products) ensures cross-shard joins are rarely needed.

HBase — At **100M sessions**, the sessions table would span multiple regions (auto-split by default at ~10GB per region). The `user_id_reverse_timestamp` row key naturally distributes across regions because user IDs are uniformly distributed. For predictable load distribution, **pre-splitting** on user_id prefix ranges (e.g., `user_000` through `user_999`) at table creation avoids initial hotspotting. Column family separation (`info`, `device`, `geo`, `activity`) becomes increasingly valuable at scale — reading only the `device` family for device analytics avoids loading large `activity:page_views` JSON blobs.

Spark — Horizontal compute scaling by adding workers to the cluster. For the current 20K-session dataset, `local[*]` mode suffices, but at scale: increase `spark.sql.shuffle.partitions` from 200 to 2,000+ for large joins (CLV cross-system join), deploy on YARN or Kubernetes for resource management, and co-locate Spark executors with HBase RegionServers for data locality on session reads.

Current limitations: Single Docker host (no distributed deployment), Spark in local mode. HBase runs natively on x86_64/WSL; Apple Silicon requires x86 emulation with significant startup latency.

8. Limitations & Future Work

Limitations:

- *Synthetic data*: Generated with Faker, producing uniform distributions without real-world seasonality or anomalies. The price elasticity analysis (Query 5) yields NULL ratios because synthetic pricing has zero variance.

- *HBase emulation*: On Apple Silicon, running `dajobe/hbase` (linux/amd64) via QEMU adds significant startup latency (5–10 minutes). Deploying on native x86_64 (WSL) eliminates this overhead. The analytics integration module includes a JSON fallback for when HBase is unavailable.
- *Single-node deployment*: Does not demonstrate true distributed processing.

Future enhancements:

- *Real-time streaming*: Spark Structured Streaming for live session event processing
- *Machine learning*: ALS collaborative filtering (Spark MLlib) for production recommendations
- *Change Data Capture*: MongoDB Change Streams to trigger incremental HBase updates

References

- MongoDB Documentation: https://www.mongodb.com/docs/
- Apache HBase Reference Guide: https://hbase.apache.org/book.html
- Apache Spark Documentation: https://spark.apache.org/docs/latest/
- happybase Documentation: https://happybase.readthedocs.io/
- PySpark API Reference: https://spark.apache.org/docs/latest/api/python/